

PROJECT DOCUMENTATION

Design & Development of a Dual-Mode Vision-Guided Robotic Arm

for Autonomous Sorting & Human-in-the-Loop Manipulation

Platform	Ubuntu 24.04 LTS
ROS2 Version	ROS2 Jazzy Jalisco
Simulation	Gazebo Harmonic
GPU	NVIDIA RTX 3060 (6GB VRAM)
Robot	6-DOF Metal Servo Arm Kit
Status	In Development — Simulation Phase

2025 – 2026

1. Project Overview

1.1 Project Title

Design and Development of a Dual-Mode Vision-Guided Robotic Arm for Autonomous Sorting and Human-in-the-Loop Manipulation

1.2 Purpose & Problem Statement

This project addresses a critical gap in modern robotic systems: the inability to seamlessly transition between fully autonomous operation and direct human control. Most industrial robotic arms operate in one fixed mode — either fully automated or manually operated — requiring physical intervention to switch between the two.

This project builds a unified system where:

- The robot operates autonomously by default — detecting, classifying, and sorting objects using computer vision without any human input.
- A human can take control at any moment simply by raising their hand in front of a camera — no buttons, no UI, no interruption.
- Remote teleoperation is possible from anywhere in the world — the human can control the arm via hand gestures streamed from a mobile phone over the internet.
- Mode switching is completely hands-free and automatic — the system detects the context and responds accordingly.

In short: the robot sorts objects by itself, but a human can take over with just a hand gesture — whether they are standing next to the robot or on the other side of the planet.

1.3 Core Objectives

- **Build and simulate a 6-DOF robotic arm in Gazebo Harmonic using ROS2 Jazzy**
- Adapt ferasboulala/five-dof-robot-arm URDF to ROS2 Jazzy
- Add 6th DOF (gripper) to the existing 5-DOF model
- Integrate ros2_control with Gazebo Harmonic for realistic simulation
- **Implement vision-based autonomous sorting**
- OpenCV color detection for fast real-time classification
- YOLOv8 (CUDA-accelerated) for advanced object detection
- Camera calibration and pixel-to-world coordinate conversion
- **Implement gesture-based control modes**
- Local gesture control: webcam captures hand, arm mirrors movement
- Remote gesture control: phone camera anywhere in world controls arm
- **Build intelligent mode management system**
- Automatic switching between AUTONOMOUS, LOCAL GESTURE, REMOTE GESTURE
- Safety timeouts and home position recovery
- **Transition simulation to real hardware using ros2_control hardware interface**

2. Hardware

2.1 Robotic Arm

Component	Details
Model	6-DOF Metal Servo Arm Kit (AliExpress)
Joints	6 Degrees of Freedom
Servos	4× MG996R + 2× YF6125MG
Frame	Black anodised aluminium brackets
Gripper	Claw-type, servo-actuated
URDF Reference	ferasboulala/five-dof-robot-arm (adapted)

2.2 Electronics

Component	Purpose
Arduino Mega	Servo PWM signal generation and serial comms
PCA9685 (16ch)	I2C servo driver board — controls all 6 servos
5V / 10A PSU	Dedicated power supply for servos
USB Cable	Serial link: Laptop → Arduino
Webcam (1080p)	Primary vision for sorting + local gesture

2.3 Development Machine

Spec	Value
OS	Ubuntu 24.04 LTS
GPU	NVIDIA RTX 3060 6GB (CUDA for YOLOv8)
Role	Brain — runs ROS2, Gazebo, vision, MoveIt2
RPi	Raspberry Pi 4/5 (planned — future phase)

Current Phase: Laptop acts as the brain. Raspberry Pi integration is planned for a future phase — the architecture is designed to support it with no code changes.

3. Software Stack

3.1 Core Infrastructure

Technology	Version / Detail	Role in Project
Ubuntu	24.04 LTS	Base operating system
ROS2 Jazzy	Jazzy Jalisco (LTS)	Robot middleware & communication
Gazebo	Harmonic (official Jazzy pair)	Physics simulation
ros2_control	Jazzy compatible	Sim ↔ Hardware abstraction layer
MoveIt2	Jazzy compatible	Motion planning & inverse kinematics
Python 3	3.10+	All node logic & vision pipeline
C++17	GCC 11+	Hardware interface plugin

3.2 Vision & AI Stack

Library	Version	Purpose
OpenCV	4.8+	Image processing, color detection, camera calibration
YOLOv8 (Ultralytics)	8.x	Object detection & classification (CUDA)
MediaPipe	0.10+	21-landmark hand tracking (gesture control)
cv_bridge (ROS2)	Jazzy	Convert ROS Image ↔ OpenCV Mat
PyTorch	2.x + CUDA 12.1	YOLOv8 GPU acceleration

3.3 Remote Control Stack

Technology	Detail	Purpose
------------	--------	---------

Technology	Detail	Purpose
MQTT	Protocol v5	Lightweight pub/sub messaging over internet
HiveMQ Cloud	Free tier cluster	Cloud MQTT broker (no server needed)
Paho MQTT (Python)	1.6+	Robot-side MQTT subscriber
Paho MQTT (JS)	Browser Websocket	Phone-side MQTT publisher
MediaPipe (Browser)	CDN via jsdelivr	Hand tracking in phone browser
Flask / Python HTTP	3.x	Serve web controller app
ngrok	Free tier	Internet tunnel for phone access

3.4 Arduino Firmware Stack

Library	Version	Purpose
Arduino IDE	2.x	Firmware development
Adafruit PWM Servo	Latest	PCA9685 I2C servo control
Wire.h	Built-in	I2C communication
pyserial (Python)	3.5+	ROS2 → Arduino serial bridge

4. System Architecture

4.1 Three Operating Modes

The system operates in one of three modes at all times, managed by a central Mode Manager node:

Mode	Trigger	Behaviour
AUTONOMOUS	No hand detected anywhere	Camera detects objects → YOLOv8 classifies → MoveIt2 picks and sorts
LOCAL GESTURE	Hand detected in webcam	Webcam → MediaPipe → joint angles → arm mirrors hand in real-time
REMOTE GESTURE	MQTT commands arriving	Phone camera → MediaPipe in browser → MQTT → ROS2 → arm moves

4.2 ROS2 Node Graph

The following nodes form the complete ROS2 computation graph:

Node	Responsibility
/camera_node	Publishes raw camera feed to /camera/image_raw

Node	Responsibility
/vision_node	YOLOv8 + OpenCV — detects & localises objects
/gesture_node (local)	MediaPipe — extracts hand landmarks from webcam
/mqtt_bridge_node	Receives remote joint commands via MQTT, publishes to ROS2
/mode_manager_node	Monitors inputs, decides active mode, publishes /control_mode
/sorting_planner_node	Maps detected object → bin → target (x,y,z) coordinates
/motion_planner_node	Movelt2 wrapper — IK + collision-free path planning
/arm_controller_node	JointTrajectoryController — executes motion via ros2_control
/arduino_bridge	Hardware interface — serial comms to Arduino + PCA9685

4.3 Remote Control Data Flow

PHONE ANYWHERE IN WORLD

- └ Browser opens web app (ngrok URL)
- └ Front camera activates
- └ MediaPipe.js detects 21 hand landmarks
- └ Maps landmarks → 6 joint angles (degrees)
- └ Publishes JSON to HiveMQ MQTT broker (SSL)

|

INTERNET (~30-80ms)

|

- └ mqtt_bridge_node.py receives JSON
- └ Converts degrees → radians
- └ Publishes JointTrajectory to /arm_controller
- └ Movelt2 executes → Arm moves
- └ Feedback sent back to phone (latency display)

Total end-to-end latency: ~230-290ms (fully usable)

5. ROS2 Package Structure

arm_ws/

|— src/

| |— my_arm_description/ ← URDF, meshes, ros2_control config

| | |— urdf/

| | |— my_arm.urdf.xacro ← Main robot description

| | |— config/

| | |— controllers.yaml ← JointTrajectoryController config

| | |— launch/

| | |— display.launch.py ← RViz with joint sliders

| | |— gazebo.launch.py ← Full Gazebo Harmonic sim

| | |— worlds/

| | |— sorting_world.sdf ← Table + colored blocks scene

| |

| |— my_arm_moveit_config/ ← Auto-generated by Setup Assistant

| | |— config/

| | |— my_arm.srdf

| | |— kinematics.yaml

| | |— controllers.yaml

| | |— launch/

| | |— moveit_rviz.launch.py

| |

| |— my_arm_vision/ ← OpenCV + YOLOv8 detection

| | |— my_arm_vision/

| | |— vision_node.py ← Color detect + YOLO classify

| | |— camera_calib.py ← Pixel → world coordinate convert

| |

| |— my_arm_gesture/ ← Local MediaPipe gesture control

| | |— my_arm_gesture/

```
| | └─ gesture_node.py
| |
| |
| └─ my_arm_remote/ ← MQTT bridge for internet control
| | └─ my_arm_remote/
| | └─ mqtt_bridge_node.py
| |
| |
| └─ my_arm_mode_manager/ ← AUTO / LOCAL / REMOTE switching
| | └─ my_arm_mode_manager/
| | └─ mode_manager_node.py
| |
| |
| └─ my_arm_hardware/ ← C++ ros2_control hardware plugin
| └─ src/
| └─ arm_hardware_interface.cpp
|
| └─ web_controller/
| └─ index.html ← Phone browser gesture controller
|
| └─ arduino/
|
| └─ arm_firmware/
| └─ arm_firmware.ino ← PCA9685 servo controller
```

PROF

6. Development Phases & Timeline

Phase	Goals	Status
Phase 1 — Workspace Setup	Install ROS2 Jazzy, Gazebo Harmonic, MoveIt2, vision libraries. Verify CUDA on RTX 3060.	In Progress
Phase 2 — URDF & RViz	Adapt ferasboulala URDF → ROS2 Jazzy Xacro. Add 6th DOF. View in RViz with joint sliders.	Planned
Phase 3 — Gazebo Sim	Launch arm in Gazebo Harmonic. ros2_control + JointTrajectoryController. Move arm via CLI commands.	Planned

Phase	Goals	Status
Phase 4 — Movelt2	Run Movelt2 Setup Assistant. Motion planning in RViz. Execute trajectories in Gazebo.	Planned
Phase 5 — Vision Pipeline	OpenCV color detection node. Camera calibration. Pixel → world coordinate conversion.	Planned
Phase 6 — Autonomous Sort	YOLOv8 integration. Full pick-sort-return cycle in simulation. Test reliability.	Planned
Phase 7 — Gesture Control	MediaPipe local gesture node. Arm mirrors hand in Gazebo. Smoothing filter.	Planned
Phase 8 — Remote Control	MQTT bridge node. Browser web app. Phone controls arm over internet.	Planned
Phase 9 — Mode Manager	Unified AUTO ↔ LOCAL ↔ REMOTE switching. Safety timeouts. Demo-ready sim.	Planned
Phase 10 — Hardware	Arduino firmware. PCA9685 wiring. ros2_control hardware interface (C++).	Future
Phase 11 — Integration	Full system on real arm. Calibration. 50+ cycle stress test.	Future
Phase 12 — RPi Migration	Move ROS2 stack to Raspberry Pi. Network-based multi-machine setup.	Future

7. Installation & Setup Commands

7.1 ROS2 Jazzy + Gazebo Harmonic Packages


```
sudo apt update
```

```
sudo apt install ros-jazzy-ros-gz ros-jazzy-ros-gz-sim ros-jazzy-ros-gz-bridge -y
```

```
sudo apt install ros-jazzy-gz-ros2-control ros-jazzy-ros2-control -y
```

```
sudo apt install ros-jazzy-ros2-controllers ros-jazzy-joint-trajectory-controller -y
```

```
sudo apt install ros-jazzy-joint-state-broadcaster -y
```

```
sudo apt install ros-jazzy-moveit ros-jazzy-moveit-setup-assistant -y
```

```
sudo apt install ros-jazzy-moveit-ros-planning-interface -y
```

```
sudo apt install ros-jazzy-xacro ros-jazzy-joint-state-publisher-gui -y
```

```
sudo apt install ros-jazzy-robot-state-publisher ros-jazzy-cv-bridge -y
```

```
sudo apt install ros-jazzy-vision-opencv ros-jazzy-camera-calibration -y
```

7.2 Python Vision Libraries (with CUDA)

```
# PyTorch with CUDA 12.1 (for RTX 3060)
```

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

```
# Vision libraries
```

```
pip install opencv-python numpy ultralytics mediapipe
```

```
# MQTT and serial
```

```
pip install paho-mqtt pyserial flask
```

```
# Verify CUDA is available
```

```
python3 -c "import torch; print('CUDA:', torch.cuda.is_available())"
```

7.3 Workspace Build

```
mkdir -p ~/arm_ws/src && cd ~/arm_ws/src  
git clone https://github.com/ferasboulala/five-dof-robot-arm.git  
cd ~/arm_ws  
rosdep install --from-paths src --ignore-src -r -y  
colcon build --symlink-install  
source install/setup.bash
```

7.4 Launch Commands

```
# View arm in RViz (joint sliders)  
ros2 launch my_arm_description display.launch.py  
  
# Full Gazebo Harmonic simulation  
ros2 launch my_arm_description gazebo.launch.py  
  
# MoveIt2 motion planning (run alongside Gazebo)  
ros2 launch my_arm_moveit_config moveit_rviz.launch.py use_sim_time:=True  
  
# Check controllers are active  
ros2 control list_controllers  
  
# Send test joint command  
ros2 topic pub --once /arm_controller/joint_trajectory </p>  
trajectory_msgs/msg/JointTrajectory </p>  
'{joint_names: [joint1,joint2,joint3,joint4,joint5,joint6], </p>  
points: [{positions: [0.785,0.0,0.5,0.0,0.0,0.0], </p>  
time_from_start: {sec: 2}}}]'
```

8. Real-World Applications

The dual-mode architecture (autonomous + human-in-the-loop) makes this system applicable across a broad range of industries. Below are the most directly relevant applications:

8.1 Manufacturing & Quality Control

- Defect sorting — vision detects faulty products, arm removes them from the line
- PCB component sorting — classify resistors, ICs, capacitors by visual markings
- Pharmaceutical tablet inspection — detect broken or discoloured tablets

8.2 Healthcare & Assistive Technology

- Lab sample sorting — sort blood vials, specimens by colour-coded labels
- Assistive robotic arm — gesture control enables mobility-impaired patients to pick up objects
- Surgical instrument pre-sorting — sterile gesture control for scrub nurses
- Medication dispensing — autonomous pill sorting for care homes

8.3 Agriculture

- Fruit grading — sort produce by size, colour, and ripeness using computer vision
- Seed sorting — separate viable seeds from damaged ones
- Harvest assistance — gesture-controlled picking for delicate crops

8.4 Logistics & E-commerce

- Package sorting — classify and route by size or barcode scan
- Returns processing — auto-classify returned items, gesture override for ambiguous cases
- Dark store fulfilment — pick individual products for delivery orders

8.5 Waste Management & Recycling

- Waste stream sorting — classify plastic, cardboard, metal by colour and shape
- E-waste disassembly — sort circuit boards, batteries, casings for recycling

8.6 Defense & Hazardous Environments

- Hazardous material handling — remote gesture control keeps humans safe from radiation/chemicals
- Bomb disposal assistance — low-cost prototype of military teleop arm systems

8.7 Education & Research

- University robotics lab — standard ROS2+MoveIt2+Gazebo teaching platform
- Human-Robot Interaction research — studying when and how humans should override autonomous systems
- Computer vision benchmarking — physical testbed for new detection algorithms

Project Demo Application: Smart Factory Sorting Station — 3 coloured blocks on a table, arm sorts them autonomously, operator raises hand to take over, steps back and arm resumes. Demonstrates all modes in a single compelling 2-minute demo.

9. Safety & Reliability Features

Safety Feature	Description
Remote Timeout	If no MQTT command arrives for 3 seconds, arm automatically returns to home position and switches to AUTO mode
Angle Clamping	All joint angles hard-clamped to safe ranges — phone cannot send values that damage servos
Smoothing Filter	Exponential moving average on gesture inputs — prevents sudden jerky movements from hand jitter
Trajectory Duration	150ms minimum per command — prevents instantaneous joint jumps
QoS 0 (No Queue)	Old commands dropped, never queued — prevents backlog of stale commands executing at once
Safe Mode Switch	Mode manager waits for current motion to complete before switching modes
Home Position	Any anomaly (disconnect, error, timeout) triggers return to predefined safe home position
Joint Limits in URDF	Physical joint limits enforced at URDF level — MoveIt2 respects them in all planning

10. Skills & Learning Path

10.1 Current Skill Level

Skill	Status
Linux Terminal	✓ Comfortable
ROS2 (Jazzy)	✓ Already experienced
Gazebo Simulation	✓ Already experienced
Python OOP & Classes	△ Needs upgrade (beginner → ROS2-ready)
OpenCV & Computer Vision	× Need to learn
YOLOv8 Object Detection	× Need to learn
MediaPipe Hand Tracking	× Need to learn
Arduino / C++ Firmware	× Need to learn (hardware phase)
ros2_control HW Interface	× Need to learn (hardware phase)
MQTT / IoT Networking	× Need to learn (remote phase)

10.2 Priority Learning Order

1. Python OOP (Week 1) — classes, callbacks, NumPy
2. OpenCV basics (Week 2) — image processing, color detection, contours
3. Camera calibration + pixel-to-world transform (Week 3)
4. YOLOv8 integration with CUDA (Week 4)
5. MediaPipe gesture mapping (Week 5)
6. MQTT remote control setup (Week 6)
7. Arduino firmware + C++ hardware interface (Hardware Phase)

11. Essential Links & Resources

11.1 Project Code References

- ferasboulala — five-dof-robot-arm (your base URDF): <https://github.com/ferasboulala/five-dof-robot-arm>
- akabot — Same arm kit with ROS2 Humble + MoveIt2 + Gazebo (reference): <https://github.com/TheNoobInventor/akabot>
- AntoBrandi — Arduinobot (best structured ROS2 arm project to study): <https://github.com/AntoBrandi/arduinobot>

11.2 ROS2 & Gazebo Documentation

- ROS2 Jazzy Documentation: <https://docs.ros.org/en/jazzy/index.html>
- Gazebo Harmonic Documentation: <https://gazebo-sim.org/docs/harmonic/getstarted>
- ros2_control Documentation: <https://control.ros.org/jazzy/index.html>
- MoveIt2 Documentation (Jazzy): <https://moveit.picknik.ai/main/index.html>
- ros_gz (ROS2-Gazebo bridge) GitHub: https://github.com/gazebo-sim/ros_gz
- URDF Tutorial (ROS2): <https://docs.ros.org/en/jazzy/Tutorials/Intermediate/URDF/URDF-Main.html>
- Xacro Documentation: <https://wiki.ros.org/xacro>

11.3 Vision & AI Libraries

- OpenCV Python Tutorials: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- YOLOv8 by Ultralytics: <https://docs.ultralytics.com>
- MediaPipe Hand Landmarker: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
- PyTorch CUDA Install Guide: <https://pytorch.org/get-started/locally/>

→ Roboflow (Custom YOLOv8 Training + Labelling): <https://roboflow.com>

11.4 Remote Control Infrastructure

→ HiveMQ Cloud (Free MQTT Broker): <https://console.hivemq.cloud>

→ Eclipse Mosquitto (Local MQTT Broker): <https://mosquitto.org>

→ Paho MQTT Python Client: <https://pypi.org/project/paho-mqtt/>

→ ngrok (Internet Tunnel — phone access): <https://ngrok.com>

→ MediaPipe JS (Browser CDN): <https://cdn.jsdelivr.net/npm/@mediapipe/hands/hands.js>

11.5 Arduino & Hardware

→ Adafruit PCA9685 PWM Servo Library: <https://github.com/adafruit/Adafruit-PWM-Servo-Driver-Library>

→ Adafruit PCA9685 Wiring Guide: <https://learn.adafruit.com/16-channel-pwm-servo-driver>

→ Arduino Serial Communication Guide: <https://docs.arduino.cc/learn/communication/uart/>

→ ros2_control Hardware Interface Tutorial:
https://control.ros.org/jazzy/doc/ros2_control/hardware_interface/doc/writing_new_hardware_component.html

11.6 Learning Resources

→ Python OOP Tutorial — Corey Schafer (YouTube): <https://www.youtube.com/watch?v=ZDa-Z5JzLYM>

→ NumPy Crash Course — Keith Galli (YouTube): <https://www.youtube.com/watch?v=GB9ByFAIAH4>

→ ROS2 Jazzy Tutorials (Official): <https://docs.ros.org/en/jazzy/Tutorials.html>

→ MoveIt2 Tutorial Series: <https://moveit.picknik.ai/main/doc/tutorials/tutorials.html>

→ Gazebo Harmonic Tutorials: <https://gazebo-sim.org/docs/harmonic/tutorials>

12. Project Backup Plans

The project has three backup plans in order of complexity. Each builds on the previous, so they are not alternatives but fallback submission options if the main project cannot be completed in time.

Backup	Title	What it Demonstrates
Backup 1 (Simplest)	Pick and Place	Basic ROS2 + MoveIt2 + fixed position pick and drop. No vision.

Backup	Title	What it Demonstrates
Backup 2	Object Sorting by Color	OpenCV color detection + autonomous sorting. No gesture control.
Backup 3	Gesture-Controlled Arm	MediaPipe local gesture control only. No autonomous mode.
Main Project	Dual-Mode Vision-Guided Arm	Full system: autonomous sorting + local gesture + remote internet control + mode switching.

Strategy: Develop in order Backup 1 → 2 → 3 → Main. Each phase is a submittable project on its own. This guarantees you always have something to present regardless of timeline.

13. Glossary of Terms

Term	Meaning
6-DOF	6 Degrees of Freedom — robot has 6 independent joint axes
ROS2	Robot Operating System 2 — middleware framework for robot software
Gazebo	Physics-based 3D robot simulator used with ROS2
URDF	Unified Robot Description Format — XML file describing robot geometry
Xacro	XML Macro language — URDF with variables and reusable macros
ros2_control	ROS2 framework abstracting hardware — same controller code for sim and real
MovelIt2	ROS2 motion planning framework — handles IK, collision avoidance, path planning
IK	Inverse Kinematics — compute joint angles given desired end-effector position
JTC	JointTrajectoryController — sends smooth position trajectories to joints
PCA9685	16-channel I2C PWM driver IC — controls up to 16 servos from one board
MQTT	Message Queuing Telemetry Transport — lightweight pub/sub IoT protocol
MediaPipe	Google framework for real-time perception — hand tracking, pose, face
YOLOv8	You Only Look Once v8 — real-time object detection neural network
CUDA	NVIDIA parallel computing platform — enables GPU-accelerated ML inference
ngrok	Secure tunnel service — exposes local server to the internet via HTTPS URL
HiveMQ	Cloud-hosted MQTT broker service — relays messages between devices
HSV	Hue-Saturation-Value — colour space better than RGB for colour detection
cv_bridge	ROS2 package converting between ROS Image messages and OpenCV Mat
QoS	Quality of Service — MQTT reliability level (0=fire-and-forget, 1=at least once)

End of Document

Dual-Mode Vision-Guided Robotic Arm — Project Documentation
Confidential — Personal Project Use Page